

# Trust & Attack the Architecture Report

Gonzalez Fernandez Jonathan Uriel – 420051871 / [jonathanuri123@gmail.com](mailto:jonathanuri123@gmail.com)  
Gabriel Hernández Yukioayax Canek – 321058526 / [yukio.gabriel.h@gmail.com](mailto:yukio.gabriel.h@gmail.com)  
Paz Rosas Jesús Eduardo – 315134854 / [jesuspazrosas@gmail.com](mailto:jesuspazrosas@gmail.com)  
Vázquez Apolonio Armando – 315140725 / [armandovaz090@gmail.com](mailto:armandovaz090@gmail.com)

## Executive Summary

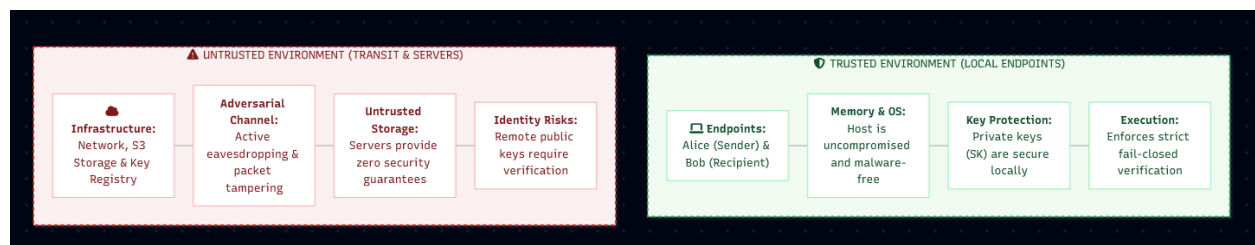
This document provides a comprehensive security assessment and threat modeling analysis for the **Secure File Exchange Platform**. The platform enables end-to-end protected file transmission across untrusted public communication channels. This report formalizes the system's trust boundaries, categorizes assets and trust assumptions, maps realistic attacker capabilities, and derives concrete design constraints from identified threat scenarios.

## 1. Identification of System Assets

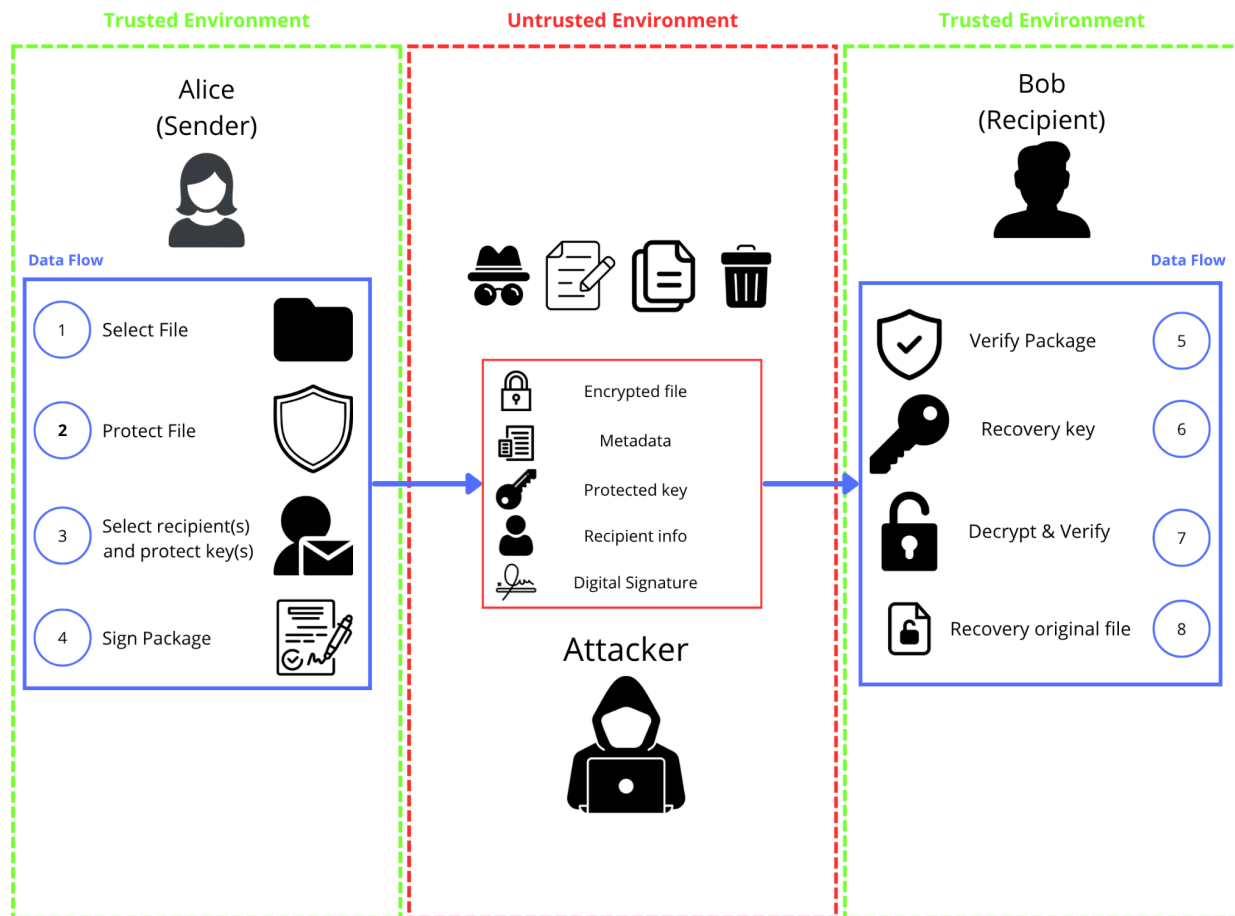
The platform processes and protects six core asset categories:

1. **Plaintext File Contents:** Primary user payload containing sensitive, proprietary, or private data requiring absolute confidentiality against unauthorized entities.
2. **File Encryption Key (FEK) / Ephemeral Session Key:** The single-use symmetric cryptographic key generated locally by the sender to encrypt the payload.
3. **Protected Metadata:** Contextual headers accompanying the file, including original filename, MIME type, file size, timestamps, transfer session IDs, and recipient public key identifiers.
4. **Long-Term Cryptographic Keys:** Private asymmetric keys used for non-repudiation, identity attestation (digital signing), and key encapsulation/decryption.
5. **Integrity & Authenticity Tokens:** Cryptographic verification tags (AEAD MAC tags and digital signatures) proving message integrity and sender origin.
6. **Recipient Ingestion Process:** The runtime environment, memory address space, and parser logic of the recipient's local machine processing untrusted input.

## 2. Component Classification & Trust Assumptions



### 3. Architecture & Trust Boundaries



#### Boundary Definitions

- **Trust Boundary 1 (Egress):** The logical perimeter between Alice's local memory/runtime and the network interface, once a package crosses this boundary, all confidentiality, integrity, and authenticity guarantees must rely solely on cryptographic transformations.
- **Trust Boundary 2 (Ingress):** The entry perimeter between the untrusted network and Bob's ingestion module, any package arriving at this boundary is treated as untrusted and potentially hostile until validated.

### 4. Attack Surface Analysis

The attack surface comprises all physical and network interfaces accessible to unauthorized entities:

1. **Serialized Package Structure:** Binary/JSON format in transit, exposing outer packet wrappers, encoding schemes, and unencrypted structural tags.
2. **Client Ingress Parser:** Deserialization libraries and buffer allocation logic in Bob's client

reading raw byte streams before cryptographic validation.

3. **Key Retrieval Protocol:** The API endpoint or transport channel querying the public key directory service.

## 5. Realistic Attacker Capabilities

- The adversary is modeled as an active network attacker positioned on the untrusted communication and storage channels with the following concrete capabilities:
- **Eavesdrop & Intercept:** Capture, monitor, and permanently store all serialized packages crossing the untrusted network or hosted on intermediate cloud storage.
- **Tamper & Bit-Flip:** Arbitrarily modify binary payloads, flip bits within the ciphertext, and alter unauthenticated header/metadata fields in transit.
- **Inject & Spoof:** Construct and inject entirely synthetic packages into Bob's ingestion interface, attempting to impersonate Alice or legitimate system entities.
- **Replay:** Capture historically valid packages and retransmit them at a later time to trigger unintended state changes or duplicate file processing.
- **Drop & Block:** Intercept and discard packages to cause Denial of Service (DoS) and prevent successful message delivery.

## 6. Concrete Attack Scenarios

- **Scenario A (Ciphertext Eavesdropping & Exfiltration):**  
Attack Path: An adversary captures the transmitted package over the untrusted network or reads it directly from the cloud relay storage.  
Objective: Extract the plaintext file payload and the symmetric File Encryption Key (FEK).  
Target Defense: Robust hybrid encryption combining symmetric AEAD with asymmetric key encapsulation.
- **Scenario B (In-Transit Payload Bit-Flipping & Tampering):**  
Attack Path: The adversary intercepts the package and alters random or targeted bits within the ciphertext and headers.  
Objective: Induce decompression errors, execute corrupted structures, or alter decrypted file contents at the recipient side.  
Target Defense: Integrity verification via AEAD authentication tags, rejecting corrupted payloads prior to decryption.
- **Scenario C (Sender Impersonation & Forged Package Injection):**  
Attack Path: An attacker crafts a malicious binary package, stamps Alice's identifier on the header, signs it with an untrusted rogue key, and routes it to Bob.  
Objective: Trick Bob into trusting and executing unverified or malicious content as if it originated from Alice.  
Target Defense: Digital signature validation against Alice's authentic, verified public key before opening the payload.
- **Scenario D (Metadata Manipulation & Path Traversal):**  
Attack Path: The attacker alters unencrypted metadata fields (e.g., modifying filename strings to include relative paths like "../") in transit.

Objective: Force the recipient client to overwrite critical local system files upon extraction.

Target Defense: Binding metadata as Additional Authenticated Data (AAD) within the AEAD envelope and applying strict filename sanitization.

- **Scenario E (Stale Package Replay Attack):**

Attack Path: The adversary captures a previously authorized, signed package and retransmits it repeatedly across Bob's ingress boundary.

Objective: Cause unintended state changes, race conditions, or duplicate file processing.

Target Defense: Enforcing monotonic sequence identifiers and short-lived expiration timestamps within the authenticated payload.

## 7. Security Mapping Table

Asset → Threat → Attack Scenario → Security Requirement → Design Constraint

| Asset                     | Threat                                                | Attack Scenario                                                                                                           | Security Requirement                | Design Constraint                                                                                                                         |
|---------------------------|-------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Plain Text File           | Interception & Eavesdropping in untrusted environment | Attacker intercepts package crossing Trust Boundary 1 or stored on an untrusted relay and attempts to read file contents. | Confidentialit                      | Protect file contents using an Authenticated Encryption with Associated Data (AEAD) symmetric scheme before leaving the trusted boundary. |
| File Encryption Key (FEK) | Unauthorized key extraction                           | Attacker extracts the FEK directly from the package wrapper to decrypt the payload without authorization.                 | Key Protection & Controlled Sharing | Encrypt the FEK using hybrid/asymmetric encryption targeted exclusively to the intended recipient's authentic public key.                 |

|                                                |                                        |                                                                                                                    |                                           |                                                                                                                                       |
|------------------------------------------------|----------------------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <b>Protected Metadata</b>                      | Tampering & parameter manipulation     | Attacker modifies metadata fields (e.g., filename, timestamps, recipient ID) while the package resides in transit. | <b>Integrity &amp; Contextual Binding</b> | Bind metadata as Additional Authenticated Data (AAD) within the AEAD scheme or include it in the cryptographically signed structure.  |
| <b>Digital Signature &amp; Sender Identity</b> | Identity Spoofing & package forgery    | Attacker creates a modified or fake package and impersonates Alice to deliver unauthorized content to Bob.         | <b>Authenticity &amp; Non-Repudiation</b> | Sign the package with the sender's private key; enforce client verification of the signature prior to decrypting or opening the file. |
| <b>Recipient Ingestion Process</b>             | Injection of malformed, corrupted data | Attacker sends a corrupted package crossing Trust Boundary 2 to exploit parsing errors or memory vulnerabilities.  | <b>Secure Failure (Robustness)</b>        | Implement fail-closed error handling and reject packages immediately if AEAD tag or digital signature verification fails.             |

## 8. Unresolved Security Questions

The following open design decisions require explicit cryptographic resolution in subsequent course modules:

### 1. **Cryptographic Composition Order:**

*Should the architecture implement "Sign-then-Encrypt" or "Encrypt-then-Sign"? What*

*are the trade-offs between leaking the sender's public identity to network eavesdroppers versus allowing the recipient to verify authorship prior to invoking key-decryption engines?*

**2. Public Key Infrastructure (PKI) & Authenticity Verification:**

*How will Alice obtain and verify Bob's authentic public key without vulnerability to Man-in-the-Middle (MitM) substitutions? Will the platform deploy hierarchical X.509 certificates, a Certificate Transparency log, or an out-of-band trust model?*

**3. Replay Defense & Freshness Guarantees:**

*What mechanism will Bob use to detect and reject replayed valid packages? Should the system use synchronized timestamps with a sliding window, monotonic counter sequences, or an ephemeral nonce registry?*

**4. Specific AEAD & Asymmetric Algorithm Selection:**

*Which exact AEAD cipher (e.g., AES-GCM, ChaCha20-Poly1305) and asymmetric scheme (e.g., RSA-OAEP, ECIES/X25519) will fulfill the design constraints regarding performance, nonce uniqueness, and post-quantum resistance?*

**5. Key Lifecycle, Revocation, and Forward Secrecy:**

*If a long-term private key is compromised, how does the system revoke trust for historical or in-transit packages, and does the architecture require ephemeral Diffie-Hellman handshakes to achieve forward secrecy?*